ХТОНИЧЕСКИЙ метод подсчёта точек при помощи Python (Ray Tracing)

В этом способе используются напрямую объекты библиотеки Tkinter, которую использует пакет turtle.

Пример задачи (из пробника Яндекса - https://education.yandex.ru/ege/variants/597a2693-1f78-45be-8daf-d80e1f937a3d/task/6):

> Исполнитель Черепаха действует на плоскости с декартовой системой координат. В начальный момент Черепаха находится в начале координат, её голова направлена вдоль положительного направления оси ординат, хвост поднят. При опущенном хвосте Черепаха оставляет на поле след в виде линии. В каждый конкретный момент известно положение исполнителя и направление его движения. У исполнителя существует четыре команды:
>
> Поднять хвост, означающая переход к перемещению без рисования;
> Опустить хвост, означающая переход в режим рисования;
> Вперёд nn (где nn — действительное число), вызывающая передвижение Черепахи на nn единиц в том направлении, куда указывает её голова;
> Направо mm (где mm — целое число), вызывающая изменение направления движения на mm градусов по часовой стрелке.
>
> Черепахе был дан для исполнения следующий алгоритм:
>
> Направо 30 Вперёд 4 Направо 330
> Опустить хвост
> Вперёд 4 Направо 90 Вперёд 7 Направо 45 Вперёд 4242
>
> Направо 135 Вперёд 11
>
> Определите, сколько точек с целочисленными координатами будут находиться внутри области, ограниченной заданными алгоритмом линиями, включая точки на линиях.

```
import turtle

# Fullscreen the canvas
screen = turtle.Screen()
screen.setup(1.0, 1.0)

# Begin!
t = turtle.Turtle()
```

```
# константа масштабирования рисунка – без масштабирования способ будет давать неправильный результат
scale = 20

# Сместимся по экрану влево и вверх
start = 11*scale
t.penup()
t.goto(-start, start)

# глобальный в рамках скрипта объект, благодаря которому работает магия автоматического подсчёта точек
canvas = t.getscreen().getcanvas()

# Каждый объект на canvas обладает своим ID. Без вникания в исходный код модуля turtle невозможно разобраться,
# какие же объекты по умолчанию создаются на canvas при запуске Черепахи, поэтому воспользуемся следующим наблюдением:
#  – даже в момент, когда перо поднято, на canvas уже присутствует объект, который будет отвечать за отображение
# следующей линии, которую нарисует Черепаха.
#  – соответственно, когда мы опускаем перо, что-то рисуем, а затем поднимается перо, всё нарисованное идёт в
# предыдущий объект, а при поднятии пера создаётся новый, с новым ID
#  – обычно этот ID самый большой из всех, что есть на canvas'е
# Соответственно, метод canvas.find_all() даёт список ID всех объектов на холсте,
# а метод max(canvas.find_all()) даёт ID объекта, в который будет отрисовываться то, что мы будем рисовать
# между следующей парой операций t.pendown() и t.penup()
# Таким образом, в списке lineIDs хранятся ID только тех линий, которые мы нарисовали Черепахой, и ничего более.
lineIDs = []
def forward(n: float):
    n = round(n * scale)
    lineIDs.append(max(canvas.find_all()))
    # Здесь мы делаем небольшой отступ между линиями, чтобы они не накладывались друг на друга.
    # Потому что нам надо, чтобы при прочерчивании линии сквозь стык двух линий рисунка из условия,
    # функция canvas.find_overlapping() выдавала только одну линию, а не две.
    t.penup()
    t.forward(3)
    t.pendown()
    t.forward(n-3)
    t.penup()
```

```
def right(n):
    t.right(n)

# mfo - My_Find_Overlapping() - обёртка над методом canvas.find_overlapping()
# метод canvas.find_overlapping(x1, y1, x2, y2) находит список ID объектов,
перекрывающихся с прямоугольником,
# натянутым на координаты (x1, y1), (x2, y2)
# смысл обёртки заключается в том, чтобы отфильтровать из найденных ID только
те, которые соответствуют
# линиям, нарисованным Черепахой по условию задания
def mfo(a, b, c, d):
    return [_ for _ in canvas.find_overlapping(a, b, c, d) if _ in lineIDs]

def check(x, y):
    # У черепахи оси координат направлены Ox вправо, а Oy вверх.
    # У холста Ox также направлена вправо, а Oy вниз.
    y = -y
    # Проверка, не находится ли точка на линии
    if len(mfo(x, y, x, y))==1:
        return True
    # Проверка, находится ли точка внутри фигуры:
    # пуск четырёх "лучей" вдоль по координатным осям
    # Если луч пересекает одну линию, то, значит, мы находимся внутри
многоугольника
    # Если луч пересекает 0 или 2 линии, значит, мы где-то снаружи
    RTX = [
        mfo(x, y, x + 50*scale, y),
        mfo(x, y, x - 50*scale, y),
        mfo(x, y, x, y + 50*scale),
        mfo(x, y, x, y - 50*scale)
    ]
    return all(len(RTX[i]) == 1 for i in range(4))

# Рисуем по командам из условия:
# Направо 30 Вперёд 4 Направо 330
# Опустить хвост
# Вперёд 4 Направо 90 Вперёд 7 Направо 45 Вперёд 4*2**0.5
# Направо 135 Вперёд 11


right(30)
t.forward(4*scale)
right(330)

forward(4)
```

```
right(90)
forward(7)
right(45)
forward(4*2**0.5))
right(135)
forward(11)

# Непосредственно подсчёт подходящих точек
counter = 0
# Фрагмент кода, необходимый для работы визуализации найденных точек
# points = []
for i in range(-15, 16):
    for j in range(-15, 16):
        x, y = i*scale, j*scale
        if check(x, y):
            counter += 1
            # Фрагмент кода, необходимый для работы визуализации найденных
точек
            # points.append((x, y))

# Вывод количества найденных подходящих точек
print(counter)

# Код для визуализации найденных точек
# t.penup()
# for point in points:
#     t.goto(point)
#     t.dot()

# Магическая строчка, необходимая для работы окна с Черепашкой
screen.mainloop()
```

Рассмотрим задачу, которую невозможно решить иначе как программным способом:

> kompege #13247
> Черепахе был дан для исполнения следующий алгоритм:
>
> Повтори 243 [Вперёд 78 Направо 120]
>
> Определите, сколько точек с целочисленными координатами будут находиться внутри области, ограниченной линией, заданной данным алгоритмом. Точки на линии учитывать не следует.

Во-первых, легко заметить, запустив простую программу (ну или подумав немного головой):

```
t = turtle.Turtle()
for _ in range(243):
    t.forward(78)
    t.right(120)
```

, что число "243" сильно избыточно: Черепаха просто рисует равносторонний треугольник, поэтому достаточно и 3 повторений.
А для нашего способа и необходимо, потому что с точки зрения canvas'a каждая новая линия, нарисованная нашей функций `forward`, поверх старой - это новый объект.

Во-вторых, в данной задаче немного другое условие: точки, попадающие на линию, мы НЕ должны учитывать. Поэтому немного перепишем функцию `check()`:

```
def check(x, y):
    y = -y
    if len(canvas.find_overlapping(x, y, x, y)) == 1:
        return False # Теперь возвращаем False в случае, если прямоугольник-точка попал на линию
    RTX = [
        mfo(x, y, x + 100 * scale, y),
        mfo(x, y, x - 100 * scale, y),
        mfo(x, y, x, y + 100 * scale),
        mfo(x, y, x, y - 100 * scale)
    ]
    return all(len(RTX[i]) == 1 for i in range(4))
```

Ну вот и всё!

Полный листинг программы:

```
import turtle

turtle.tracer(False) # Для отключения анимации Черепахи
screen = turtle.Screen()
screen.setup(1.0, 1.0)

# Begin!
t = turtle.Turtle()

scale = 1000000 # O_o ################################
start = 30*scale
t.penup()
t.goto(-start, start+100)
```

```
canvas = t.getscreen().getcanvas()
lineIDs = []


def forward(n):
    n = n * scale
    lineIDs.append(max(canvas.find_all()))
    t.penup()
    t.forward(3)
    t.pendown()
    t.forward(n - 3)
    t.penup()


def right(n):
    t.right(n)


def mfo(a, b, c, d):
    return [_ for _ in canvas.find_overlapping(a, b, c, d) if _ in lineIDs]


# Функция проверки того, находится ли точка с данными координатами внутри
фигуры или на её границе
def check(x, y):
    y = -y
    if len(canvas.find_overlapping(x, y, x, y)) == 1:
        return False
    RTX = [
        mfo(x, y, x + 100 * scale, y),
        mfo(x, y, x - 100 * scale, y),
        mfo(x, y, x, y + 100 * scale),
        mfo(x, y, x, y - 100 * scale)
    ]
    return all(len(RTX[i]) == 1 for i in range(4))


for _ in range(3):
    forward(78)
    right(120)

print("End of drawing")

counter = 0
for i in range(-100, 100):
    for j in range(-100, 100):
```

```
            x, y = i * scale, j * scale
            if check(x, y):
                counter += 1

    print(counter)

    screen.mainloop()
```

Видели этот scale?
Я последовательно увеличивал это значение, и каждый раз у меня получалось разное количество точек.
Где-то при переходе от 1000 к 10000 оно начало расти довольно медленно.
Наконец, при переходе от 100000 к 1000000 не произошло никакого изменения.

Из чего я делаю вывод, что правильный ответ к этой задаче 2672, а не 2595.
А дело всё в том, что вычисления с плавающей точкой - штука весьма капризная и неточная, и если брать слишком малую точность вычислений, то может так статься, что точки сольются в линию. Хотя на самом деле это может быть далеко не так.